Lab 1: Platform Baseline & Roofline Bootstrapping

November 9, 2026

1 Introduction

This is the first in a series of four labs in the Sustainable Computing course. Throughout the lab series, you will gain hands-on experience with sustainable computing by developing energy-efficient applications on the Raspberry Pi 5. You will learn how to characterize the platform, manage and optimize power–performance trade-offs, and combine the learned techniques to meet application requirements efficiently.

Maximizing energy efficiency, that is, reducing the energy required to perform a given amount of work, is a main pillar of sustainable computing. A prerequisite for achieving high energy efficiency is understanding how an application utilizes the available system resources and identifying the bottlenecks that limit its performance. This characterization provides the foundation for selecting effective optimizations.

In this lab, you will use the **Roofline model** to identify application performance bottlenecks and understand the performance limits of the Raspberry Pi 5. The Roofline model is an essential tool for performance engineers and system architects seeking to understand and optimize application performance on a given hardware platform. It is a visual performance model that helps identify whether an application is limited by computational capability or memory bandwidth. These insights can then guide programmers and architects toward optimizations that address the actual performance bottleneck.

In this lab you will learn:

- How to build a Roofline model using microbenchmarks.
- Analyze a set of kernels using the Roofline model to identify performance bottlenecks and potential optimizations.
- Apply the Roofline model to study multithreaded applications.

2 Lab preparation and general guidelines

- Take a look at "**Roofline: an insightful visual performance model for multicore architectures**"; focus on the high-level idea, not every detail.
- Familiarize yourself with the concept of **Operational Intensity (OI)**. OI is calculated as the ratio of the number of operations performed by a kernel to the amount of data transferred between the processor and main memory. In this lab, we consider **floating-point operations**, and we assume an **ideal cache behavior** and consider **only compulsory memory traffic**, ignoring additional traffic caused by capacity and conflict misses. Both **loads and stores** are included in the calculation. To obtain OI of a kernel, count the floating-point operations performed by the kernel, count the bytes that must be loaded from and stored to main memory, and calculate

$$OI = \frac{\text{Total number of floating-point operations}}{\text{Total number of bytes transferred to and from main memory}}$$

- The lab is divided into a series of tasks, each consisting of several sub-tasks. After each sub-task, or group of related sub-tasks, you will find a section titled **In the lab report**, which describes what you need to include in the report template (`Lab1_report_template.docx`) provided on the lab page in Canvas. Therefore, before starting each task, it is recommended that you read the relevant sub-task instructions and the corresponding **In the lab report** section in full.

UNIVERSITY OF GOTHENBURG

CHALMERS UNIVERSITY OF TECHNOLOGY

- The lab consists of two types of activities: experimental tasks, in which you run experiments and report the results, and analysis questions, in which you interpret and reason about the results obtained. Since the **Raspberry Pis are available only during the scheduled lab sessions, we strongly recommend completing all experimental tasks first**. You should aim to complete the entire lab during the scheduled sessions. However, if any analysis questions remain after you have completed the experimental tasks, you may finish them afterward at your own pace.
- All required code and scripts are available in the directory `/home/dat278/labs/lab1` on the Raspberry Pi 5. A backup directory, `/home/dat278/labs/lab1_template`, is located beside it and contains the original versions of the lab files. If you modify a file and wish to restore its original version, you may copy the corresponding file from this directory. **Do not modify, move, or delete any files in `/home/dat278/labs/lab1_template`.**
- After finishing the lab tasks, you **need to run** `encrypt_and_update_lab_files.sh`, call a TA to record your attendance, and make sure that the script ran successfully. This is a **necessary condition for passing the lab**.
- **Important: If you want to keep a copy of your work, make sure to copy it to your own device before running the encryption script. After encryption, the original files may not be recovered.**

3 In lab tasks

The following sections describe a set of tasks that you should perform during the lab. All required code and scripts can be found in the directory `/home/dat278/labs/lab1` on the Raspberry Pi 5. All the directory paths listed in the rest of the document are found inside that directory.

3.1 Task 1

In this task, you will use two benchmarks to draw the flat and slanted roofs of the Roofline model that represent the peak compute and memory bandwidth, respectively. The steps are as follows:

1. Use the **two** benchmarks (described below) under `roofline_emp` to obtain the sustained peak DRAM bandwidth and the compute peak of a single scalar core of the Raspberry Pi 5 and record these values in `inp/hw_specs.json`. For example, if the obtained bandwidth is 5.9 GB/s and the peak compute is 7.2 GFLOP/s, the file content should be:

   ```
   {
   "bandwidth_gb_s":  5.9,
   "peak_gflops":  7.2
   }
   ```

   The **two** benchmarks are:

   - **Stream**: This benchmark is used to obtain the sustained peak DRAM bandwidth. The script `run_stream_bench.sh` compiles and runs the benchmark and outputs the `Sustained memory bandwidth (Triad)`. To run the script:

     ```
     chmod +x ./run_stream_bench.sh
     ./run_stream_bench.sh
     ```

   - **Scalar_roof**: This benchmark is used to obtain the `Peak GFLOP/s` using a single scalar core. The script `run_peak_flops.sh` compiles and runs the benchmark. To run the script:

     ```
     chmod +x ./run_peak_flops.sh
     ./run_peak_flops.sh
     ```

2. Once peak performance and bandwidth are obtained, you can use the roofline_emp/draw_roofline.py script to draw the roofs. The output of the script (a Roofline figure) is stored in out directory.

In the lab report (Lab1_report_template.docx):

- Report the values of the peak performance, memory bandwidth, and the ridge point coordinates in the obtained figure.
- Report the minimum Operational Intensity (OI) for a kernel to be categorized as compute-bound using the obtained Roofline.

3.2 Task 2

In this task, you will analyze a set of kernels using the Roofline model.

1. In comp-b_vs_mem-b/kernels.c, you have implementations of four kernels:
   - **Triad:** Performs $a[i] = b[i] + c[i] * d[i]$ for each element.
   - **Dot Product:** Computes the sum of element-wise products of two vectors.
   - **FMA Chain:** Performs repeated fused multiply-add operations $x = x * y + c$ on each element with multiple independent accumulators.
   - **Polynomial Evaluation (Horner):** Evaluates a polynomial on each element using Horner's method with multiple independent accumulators.

   Run these four kernels and plot the results on the Roofline by following these steps:

   (a) **The kernels' operational intensity (OI):** As described in the Roofline paper, the OI of a kernel can be obtained by coupling the 3Cs model to the Roofline model. As we described in Section 2, we here consider only the compulsory misses, hence the best possible OI. To get the OI for each kernel using compulsory misses, examine the code to identify the total number of operations and the number of bytes loaded and stored (Section 2). Use the default problem sizes from run_kernels.sh to calculate the OI. **Hint: In cases where there are unrolled loops, it may be easier to infer the OI from the parts that have no unrolling.**

   (b) **The achievable throughput:** You need to run the kernel on the Raspberry Pi to get the achieved throughput.

   ```
   chmod +x ./run_kernels.sh
   ./run_kernels.sh
   ```

   (c) Record the OI and achievable throughput values in inp/kernel_specs.json.

   ```
   {
   "fma":  {"measured_gflops":  0.1, "operational_intensity":  1},
   "dot":  {"measured_gflops":  1.2, "operational_intensity":  2},
   "poly":  {"measured_gflops":  3.4, "operational_intensity":  3},
   "triad":  {"measured_gflops":  4.5, "operational_intensity":  4}
   }
   ```

   (d) After recording the values in inp/kernel_specs.json, run roofline_emp/draw_roofline.py.

   In the lab report (Lab1_report_template.docx):

   (a) List the OIs and achievable throughput of the four kernels.

   (b) For which of the four kernels is the OI dependent on the problem size, and for which is it independent?

   (c) Categorize the four kernels into compute-bound and memory-bound.

(d) Report the "ratio to roof" achieved by the four kernels. These values can be seen on the Roofline figure you generated.

(e) What does the ratio to roof of a kernel indicate?

(f) For which kernels is the ratio to roof greater than 1? why?

2. In matmul directory, you have five implementations of a matrix multiplication kernel with different levels of optimization. Examine the kernels and calculate their OIs (Use the default problem sizes from run_matmul.sh to calculate the OI), and then run these kernels using matmul/run_matmul.sh by providing the kernel version as in the example below, update the inp/kernel_specs.json and plot the results on the Roofline.

```
# Example:  to run version 1 (matmul_1)
chmod +x ./run_matmul.sh
./run_matmul.sh 1
```

3. Now run the experiments (using run_matmul.sh) with a matrix_size of 512, then 1024.

```
# Example:  to run version 1 (matmul_1) with size 512
./run_matmul.sh 1 512
```

4. Take a look at the list of kernel optimizations in Table 3 in the Roofline paper.

In the lab report (Lab1_report_template.docx):

1. Take the matmul_1 kernel as a baseline and for the rest, compare the throughput obtained in each case with matmul_1. Using both the kernel codes and the list of kernel optimizations in Table 3 in the Roofline paper, identify which of these optimizations, if any, is/ are applied. Report your observations using a format similar to the following:

   (a) matmul_1 vs. matmul_2: throughput improvement 1.5×. Applied optimizations: memory affinity and software prefetching.

   (b) Report the results using both sizes of 512 and 1024.

2. Which version achieved the best performance? What is the ratio of the performance to the roof? Optional: can you think of other optimizations to further improve the performance?

3.3 Task 3

In the previous tasks, you were introduced to the Roofline model and learned how to measure the compute and memory roofs of a system, as well as how to use the model to analyze the performance of applications and identify potential bottlenecks. The Roofline model constructed in the previous tasks represents the performance limits of a single core. In this task, we extend the analysis to multicore execution and investigate how performance scales when multiple cores are utilized. To this end, we will use the OpenMP library for shared-memory parallel programming (see the OpenMP reference guides for additional information).

1. Measure the multicore compute roof, navigate to multicore directory and run run_peak_multicore.sh. This command provides the multicore peak compute performance, which you will use to draw the new roofline. The sustained memory bandwidth you measured earlier remains unchanged.

2. Navigate to multicore/kernels_omp and run ./run_kernels_mt.sh. This script runs the multicore versions of the kernels used in Task 2.1.

   In the lab report (Lab1_report_template.docx):

   (a) List the Operational Intensities (OI) and achieved throughput for the four kernels and the "ratio to roof" (you need to modify the JSON files and draw the multicore Roofline as you did in the single-core tasks).

UNIVERSITY OF GOTHENBURG

CHALMERS UNIVERSITY OF TECHNOLOGY

(b) Report which kernels transitioned from being compute-bound to memory-bound and vice versa when moving from the single-core to multicore case.

3. In the previous tasks, you executed the kernels with the default number of iterations (200) and default size (1,000,000). These defaults are set at the beginning of `run_kernels_mt.sh` for the multicore version and `lab1/comp-b_vs_mem-b/run_kernels.sh` for the single-core version. Now, set the number of iterations (ITERS) to 30 and execute both the single-core and multicore versions.

   In the lab report (Lab1_report_template.docx):

   (a) Report which kernels transitioned from being compute-bound to memory-bound and vice versa when moving from 200 to 30 iterations on single-core and multicore cases.

   (b) For kernels that are compute-bound on single-core and multicore, would you generally expect the ratio to the roof to stay the same, to increase, or to decrease when moving from single-core to multicore? Why (in one or a few words)?

   (c) A kernel K is compute-bound on system SC and memory-bound on system SB. Describe two scenarios in which its achieved performance on system SC could still be lower than on system SB.